MINOR PROJECT REPORT · 2025–26

CareerBuilder

AI-Powered Dual-Sided Job Matching Platform

Project Title	CareerBuilder – Semantic AI Job Matching System
Submitted By	Sharan SP
Email	sharan.sp95@gmail.com
Course	B.E. Computer Science & Engineering – 6th Semester
Project Type	Minor Project (Full-Stack + ML)
Report Date	May 2026
Domain	Artificial Intelligence · Natural Language Processing · Web Engineering

CONFIDENTIAL · ACADEMIC USE ONLY

Table of Contents

1. Project Summary

What is CareerBuilder?

CareerBuilder is a full-stack, AI-powered job-matching web application that bridges the gap between job seekers and recruiters in the Indian tech industry. It uses *Natural Language Processing (NLP)* — specifically BERT sentence embeddings — to intelligently rank job listings against a candidate's resume, going far beyond traditional keyword-based Applicant Tracking Systems (ATS).

Problem It Solves

For Job Seekers

- Traditional portals use keyword search, missing semantic meaning
- Candidates don't know which skills they're lacking
- No unified place to track application progress
- Generic job listings with no relevance scoring

For Recruiters

- Manually reviewing hundreds of resumes is time-consuming
- Hard to identify best-fit candidates quickly
- No tool to quantify resume-to-JD match quality
- Scattered candidate data across multiple systems

Solution

CareerBuilder provides a **dual-sided platform**:

✓ Job seekers upload a PDF resume → AI extracts skills and ranks matching jobs with percentage scores

✓ Recruiters paste a job description → AI ranks all registered candidates by fit

✓ A Kanban board helps job seekers track their application pipeline

✓ A conversational AI chatbot (CareerBot) provides personalized career coaching

✓ Skill gap analysis identifies what candidates should learn next

Project Scope: This is a complete, production-ready web application with a Next.js frontend, Flask REST API backend, SQLite database, BERT-based ML model, Google OAuth, email notifications, and an AI chatbot — built as a 6th-semester minor project.

2. Tech Stack

Frontend

FRAMEWORK	UI LIBRARY	LANGUAGE
Next.js v16.2.3 (Turbopack)	**React** v19.2.4	**TypeScript** Strict Mode
STYLING	**ANIMATIONS**	**HTTP CLIENT**
Tailwind CSS v4 (PostCSS)	**Framer Motion** v12.38.0	**Axios** v1.15.0
AUTH	**DRAG & DROP**	**CHARTS**
NextAuth.js v5.0.0-beta.31	**DnD Kit** Core v6.3.1	**Recharts** v3.8.1
FILE UPLOAD	**ICONS**	**TESTING**
React Dropzone v15.0.0	**Lucide React** v1.8.0	**Jest + ts-jest** Latest

Backend

FRAMEWORK	LANGUAGE	ML / NLP
Flask v3.1.0 (Python)	**Python** 3.10+	**Sentence-Transformers** v3.4.1

ML MODEL **all-MiniLM-L6-v2** BERT (384-dim)	SIMILARITY **scikit-learn** v1.6.1	PDF PARSING **PyPDF2** v3.0.1
DATABASE **SQLite** Built-in Python	AUTH TOKENS **PyJWT** v2.10.1	OAUTH **google-auth** v2.38.0
RATE LIMITING **Flask-Limiter** v3.9.0	COMPRESSION **Flask-Compress** v1.15	TESTING **Pytest** Latest

External Services & AI APIs

AI CHATBOT **Groq API** Llama / Gemma models	ALT LLM **OpenRouter API** Fallback	AUTH PROVIDER **Google OAuth 2.0** openid scope
EMAIL **Gmail SMTP** Port 587 TLS	JOB DATA **LinkedIn / Indeed / Internshala** Scraped	STATIC JOBS **jobs.json** ~10,000 listings

3. Folder Structure

Root Directory

```
minor project Copy/
├── backend/ # Flask Python backend
├── src/ # Next.js frontend (React 19 + TypeScript)
├── public/ # Static assets (favicon, images)
├── node_modules/ # npm packages (auto-generated)
├── .next/ # Next.js build output (auto-generated)
├── package.json # Frontend dependencies and scripts
├── next.config.ts # Next.js configuration (Turbopack, images)
├── tsconfig.json # TypeScript compiler options
├── tailwind.config.js # Tailwind CSS customizations
├── jest.config.js # Jest unit test configuration
├── .env.local # Frontend environment secrets (gitignored)
└── README.md # Project documentation
```

Backend Directory (`backend/`)

```
backend/
├── app.py # Flask app + all API route handlers (30+ endpoints)
├── auth.py # JWT auth, user registration/login, DB schema (6 tables)
├── model.py # BERT-based job ranking + skill extraction logic
├── scraper.py # Live job scraping (LinkedIn, Indeed, Internshala)
├── users.db # SQLite database file
├── jobs.json # Static job dataset (~10,000 listings)
├── scraped_jobs.json # Cached live job scraping results
├── requirements.txt # Python package dependencies
├── .env # Backend secrets (API keys, SMTP, JWT)
└── tests/
    ├── conftest.py # Pytest fixtures and setup
    ├── test_api.py # API endpoint integration tests
    ├── test_auth.py # Authentication unit tests
    └── test_model.py # ML model ranking tests
```

Frontend Source (`src/`)

```
src/
├── app/ # Next.js App Router pages
│ ├── page.tsx # Home / landing page
│ ├── layout.tsx # Root layout (Navbar, providers, AuthSync)
│ ├── globals.css # Global styles (Tailwind v4 + animations)
│ ├── api/auth/[...nextauth]/
│ │ └── route.ts # NextAuth API route (Google OAuth handler)
│ ├── login/page.tsx # Login page (email + Google tabs)
│ ├── register/page.tsx # 3-step registration wizard
│ ├── select-role/page.tsx # Role selection (job_seeker / recruiter)
│ ├── forgot-password/page.tsx # Password reset request
│ ├── reset-password/page.tsx # Password reset via token link
│ ├── verify-email/page.tsx # Email verification landing
│ ├── dashboard/page.tsx # Job seeker analytics dashboard
│ ├── upload/page.tsx # PDF resume upload + parsing
│ ├── jobs/page.tsx # AI job recommendations listing
│ ├── tracker/page.tsx # Kanban application tracker
│ ├── skill-gap/page.tsx # Skill gap visualization
│ ├── notifications/page.tsx # Recruiter shortlist notifications
│ ├── profile/page.tsx # User profile editor
│ ├── recruiter/page.tsx # Recruiter portal (search + analytics)
│ └── unauthorized/page.tsx # 403 access denied
├── components/ # Reusable React components
│ ├── Navbar.tsx # Navigation bar (desktop + mobile drawer)
│ ├── AuthGuard.tsx # Route protection HOC
│ ├── AuthSync.tsx # Syncs Google OAuth token with backend JWT
│ ├── Chatbot.tsx # CareerBot AI chat modal
│ ├── JobCard.tsx # Job listing card with match score
│ └── Toast.tsx # Toast notification system
└── lib/ # Utility functions and services
    ├── api.ts # Axios HTTP client (JWT interceptor)
    ├── auth.ts # Auth helpers (login, logout, token storage)
    ├── tracker.ts # Kanban board API helpers
    ├── job-meta.ts # Job type / salary range metadata
    ├── bookmarks.ts # Job bookmarking utilities
    └── theme.tsx # Dark/light mode ThemeProvider
```

4. Core Features

Job Seeker Features

Feature	Description	Technology
AI Resume Parsing	Upload a PDF resume; the system extracts text and identifies 70+ technical skills automatically	PyPDF2, regex skill extraction
Semantic Job Matching	Ranks jobs by AI match score (0–100%) using BERT embeddings and skill overlap analysis	Sentence-Transformers (all-MiniLM-L6-v2), cosine similarity
Match Score Breakdown	Shows matched skills (green), missing skills (red) per job listing with visual indicators	Hybrid scoring: 70% skill overlap + 30% semantic similarity
Skill Gap Analysis	Analyzes missing skills across all job recommendations and visualizes demand frequency	Recharts, aggregated missing skills
Kanban Application Tracker	Drag-and-drop board with 4 columns: Saved → Applied → Interview → Offer/Rejected	DnD Kit, SQLite persistence
Analytics Dashboard	Charts: top demanded skills (bar), skill gaps (horizontal bar), job modality (pie), KPI cards	Recharts, computed from recommendations
CareerBot AI Assistant	In-app chat for resume tips, interview prep, job search strategy with conversation history	Groq API (Llama/Gemma), Flask backend
Recruiter Notifications	Receive in-app + email notifications when a recruiter shortlists you	SQLite, Gmail SMTP
Profile Management	Edit name, email, view resume skills, change password, download resume	REST API, localStorage

Dark / Light Mode	Persistent theme toggle with system preference detection	ThemeProvider context, localStorage

Recruiter Features

Feature	Description	Technology
AI Candidate Ranking	Paste a job description; system ranks all platform candidates by fit score (0–100%)	BERT embeddings, 90% skill + 10% semantic scoring
One-Click Shortlisting	Shortlist a candidate; triggers in-app notification + email to the candidate	Gmail SMTP, SQLite notifications
Recruiter Analytics	View KPIs: total shortlisted, unique skills in pool, top skill distribution charts	Recharts, aggregated from shortlist DB
Shortlisted Candidates	View all shortlisted candidates with contact information and resume details	REST API, SQLite
Pagination	Candidate results paginated (up to 50 per page) for large candidate pools	Flask pagination, frontend state

Authentication Features

✓ Email/password registration with strength indicator and confirmation

✓ Google OAuth 2.0 single sign-on

✓ Email verification via tokenized link (15-minute expiry)

✓ Forgot password / reset password via email

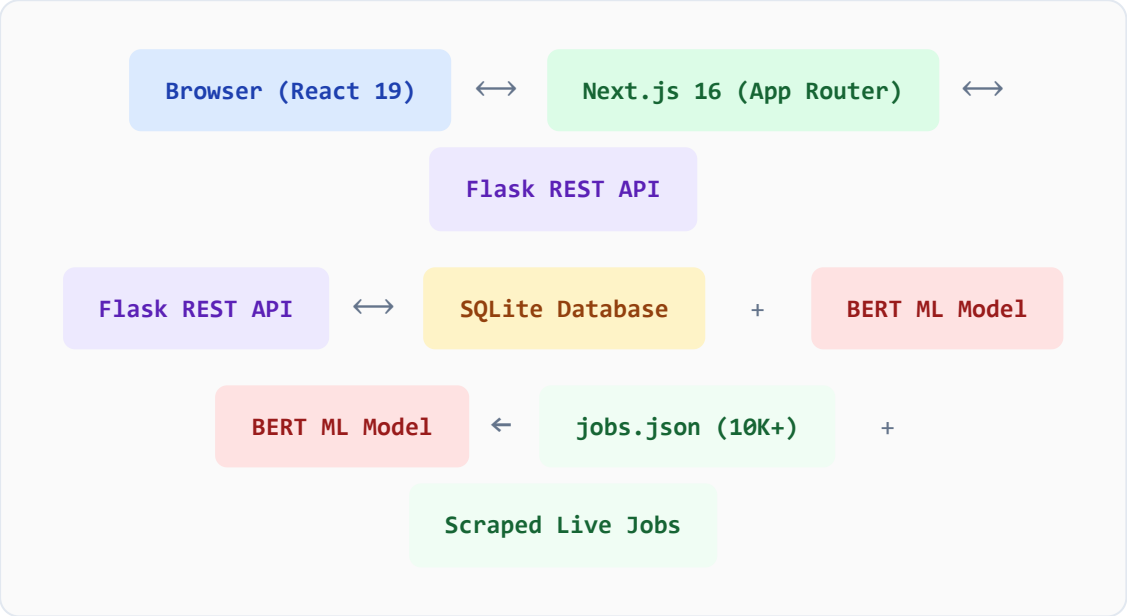
✓ Role-based access control (job_seeker vs recruiter)

✓ JWT tokens with 7-day expiry, stored in HttpOnly cookies

✓ Route protection with AuthGuard HOC

5. How It Works – Architecture & Data Flow

High-Level Architecture



Job Seeker Flow – Step by Step

1. **Register / Login** — User signs up with email or Google OAuth. Backend issues a JWT (7-day expiry), stored in localStorage + HttpOnly cookie.

2. **Upload Resume** — User uploads a PDF (max 5MB). PyPDF2 extracts raw text. Regex pattern matches 70+ skills. Skills + text saved to DB.

3. **AI Job Matching** — Axios sends `POST /api/recommend`. Flask encodes the resume using BERT (all-MiniLM-L6-v2). Computes cosine similarity against pre-computed job embeddings. Returns top matches with match %, matched skills, missing skills.

4. **Browse Jobs** — Jobs page renders AI-ranked JobCards sorted by match score. User can filter, apply externally, or save to tracker.

5. **Application Tracker** — Saved jobs appear in "Saved" column. User drags cards across Kanban columns (Applied → Interview → Offer). State persisted to `kanban_cards` table.

6. **Skill Gap Analysis** — Frontend aggregates `missingSkills` from all recommendations, sorts by frequency, renders bar charts.

7. **CareerBot Chat** — User messages sent to `POST /api/chat`. Flask calls Groq API with conversation history + career coaching system prompt. Response streamed back to chat UI.

Recruiter Flow – Step by Step

1. **Login as Recruiter** — User selects "Recruiter" role during registration. Route guard enforces role-based access to `/recruiter`.

2. **Candidate Search** — Recruiter pastes job description. `POST /api/recruit_candidates` loads all registered candidate resumes from DB.

3. **AI Ranking** — Backend encodes both the job description and each candidate's resume with BERT. Candidates ranked by: **90% skill coverage + 10% semantic similarity**.

4. **Shortlist** — Recruiter clicks "Shortlist". `POST /api/shortlist` inserts a notification record in DB and sends an email to the candidate via Gmail SMTP.

5. **Analytics** — Recruiter dashboard shows KPI cards and charts from all shortlisted candidates.

ML Scoring Formula

Job-Seeker Match Score (used in `/api/recommend`):

```
match_score = (0.70 × skill_overlap_ratio) + (0.30 × cosine_similarity)
            × 100  →  rounded to integer [0, 100]

Where:
  skill_overlap_ratio = matched_skills / total_job_skills
  cosine_similarity   = BERT_embed(resume) · BERT_embed(job_description)
```

Recruiter Candidate Ranking (used in `/api/recruit_candidates`):

```
candidate_score = (0.90 × candidate_skill_coverage) + (0.10 × cosine_simil
            × 100  →  rounded to integer [0, 100]
```

Authentication Flow

Email Auth

1. Register → backend hashes password (werkzeug Argon2/PBKDF2)
2. Verification email sent with 15-min JWT link
3. User clicks link → backend marks verified, returns session JWT
4. Frontend stores JWT in localStorage + HttpOnly cookie

Google OAuth

1. NextAuth redirects user to Google consent screen
2. Google returns ID token to NextAuth callback
3. AuthSync component exchanges token with Flask `/api/auth/google`
4. Backend verifies with `google.oauth2.id_token`, issues JWT

6. APIs & Endpoints

Health & Utility

Method	Endpoint	Purpose	Auth
GET	`/`	Health check	None
GET	`/health`	Detailed health status	None
POST	`/api/dev/reset-db`	Reset database (dev mode only)	Dev

Authentication

Method	Endpoint	Purpose	Auth
POST	`/api/auth/register`	Email + password registration with role	None
POST	`/api/auth/login`	Login with email and password, returns JWT	None
POST	`/api/auth/google`	Exchange Google ID token for platform JWT	None
POST	`/api/auth/forgot-password`	Send password reset link via email	None
POST	`/api/auth/reset-password`	Reset password using reset token	None
GET	`/api/auth/verify-email-link`	Verify email via tokenized link	None
POST	`/api/auth/set-role`	Set or change user role (job_seeker/recruiter)	JWT
POST	`/api/auth/logout`	Clear auth cookie, logout user	JWT

Profile & Resume

Method	Endpoint	Purpose	Auth
GET	`/api/user/profile`	Get user info, uploaded resume, extracted skills	JWT
POST	`/api/upload`	Upload PDF resume; parse text + extract skills (10/hr limit)	JWT

Job Recommendations

Method	Endpoint	Purpose	Auth
POST	`/api/recommend`	AI job recommendations (30/min limit). Accepts resume_text, skills, source (static/live/hybrid)	JWT
GET	`/api/recommendations/cached`	Retrieve last cached recommendation results for user	JWT

Application Tracker (Kanban)

Method	Endpoint	Purpose	Auth
GET	`/api/tracker`	Get full Kanban board (all columns + jobs)	JWT
POST	`/api/tracker`	Add job to "Saved" column	JWT
PATCH	`/api/tracker/<job_id>`	Move job to different Kanban column	JWT
DELETE	`/api/tracker/<job_id>`	Remove job from tracker	JWT

Notifications

Method	Endpoint	Purpose	Auth
GET	`/api/get_notifications`	Get all recruiter shortlist notifications for user	JWT
PATCH	`/api/notifications/<notif_id>/read`	Mark a notification as read	JWT

Recruiter Endpoints

Method	Endpoint	Purpose	Auth
POST	`/api/recruit_candidates`	Rank all candidates against a job description (pagination: page, page_size≤50)	JWT + Recruiter role
POST	`/api/shortlist`	Shortlist a candidate; send in-app + email notification	JWT + Recruiter role
GET	`/api/recruiter/shortlisted`	List all shortlisted candidates by this recruiter	JWT + Recruiter role

AI Chat

Method	Endpoint	Purpose	Auth
POST	`/api/chat`	CareerBot AI assistant (20 msg/min limit). Accepts message + conversation history (last 20 turns)	JWT

Standard Response Format

```
{
  "success": true,
  "message": "Operation description",
  "data": {
    // endpoint-specific payload
  }
}
```

Rate Limiting Summary

Endpoint	Limit
`/api/upload`	10 uploads per hour per user
`/api/recommend`	30 requests per minute

`/api/chat`	20 messages per minute

7. Database & Storage

The application uses **SQLite** (`backend/users.db`) with **6 tables**, managed entirely through raw SQL in `auth.py`. The schema is initialized on first run and supports live migration (adding new columns without data loss).

users

`id`	INTEGER PK AUTOINCREMENT
`name`	TEXT NOT NULL
`email`	TEXT UNIQUE NOT NULL
`password_hash`	TEXT NOT NULL
`google_id`	TEXT (nullable)
`role`	TEXT DEFAULT NULL
`is_verified`	INTEGER DEFAULT 1
`verified`	INTEGER DEFAULT 1
`created_at`	DATETIME DEFAULT NOW

resumes

`id`	INTEGER PK AUTOINCREMENT
`user_id`	INTEGER NOT NULL UNIQUE → users.id
`resume_text`	TEXT NOT NULL (raw PDF text)
`skills`	TEXT NOT NULL (JSON array)
`file_name`	TEXT (original filename)
`uploaded_at`	DATETIME DEFAULT NOW

kanban_cards

`id`	INTEGER PK AUTOINCREMENT
`user_id`	INTEGER NOT NULL → users.id
`job_id`	INTEGER NOT NULL
`title`	TEXT NOT NULL
`company`	TEXT NOT NULL
`location`	TEXT DEFAULT ''
`match_score`	INTEGER DEFAULT 0
`col`	TEXT DEFAULT 'Saved'
`added_at`	DATETIME DEFAULT NOW
`UNIQUE`	(user_id, job_id)

notifications

`id`	INTEGER PK AUTOINCREMENT
`user_id`	TEXT NOT NULL → users.id
`message`	TEXT NOT NULL
`company`	TEXT
`job_title`	TEXT
`required_skills`	TEXT
`recruiter_email`	TEXT
`recruiter_name`	TEXT
`is_read`	INTEGER DEFAULT 0
`created_at`	DATETIME DEFAULT NOW

ai_conversations

`id`	INTEGER PK AUTOINCREMENT
`user_id`	INTEGER NOT NULL → users.id
`user_message`	TEXT NOT NULL
`ai_response`	TEXT NOT NULL
`created_at`	DATETIME DEFAULT NOW

recommendation_cache

`user_id`	INTEGER PK → users.id
`data`	TEXT NOT NULL (JSON blob)
`updated_at`	DATETIME DEFAULT NOW

Stores: {recommendations, source, keyword}. One row per user, upserted on each recommendation call.

Static Data Storage

File	Contents	Size
`backend/jobs.json`	Static job listings with title, company, location, description, required_skills, apply_url	~10,000 jobs
`backend/scraped_jobs.json`	Cached live jobs scraped from LinkedIn / Indeed / Internshala	Variable

Database Migration Strategy

The `migrate_db()` function in `auth.py` runs on every server start. It uses `ALTER TABLE ... ADD COLUMN` statements inside `try/except` blocks to safely add new columns without losing existing data. Deprecated tables are dropped automatically.

8. Authentication & Security

Authentication Mechanisms

Mechanism	Implementation	Details
JWT Tokens	PyJWT (HS256)	7-day expiry. Contains: user_id, email, name, role, iat, exp. Stored in localStorage + HttpOnly cookie.
Password Hashing	werkzeug.security	PBKDF2/Argon2 hashing. Passwords never stored in plaintext.
Google OAuth 2.0	google-auth, NextAuth.js v5	ID token verified server-side with Google's public keys. No client-side trust.
Email Verification	JWT link (15-min expiry)	Registration sends tokenized link. Backend validates and marks user as verified.
Password Reset	JWT link (15-min expiry)	Secure token-based reset. Short expiry limits attack window.
Role-Based Access Control	`@require_role` decorator	Backend enforces recruiter-only endpoints. Frontend AuthGuard matches `requiredRole`.

Security Controls Implemented

✓ CORS restricted to specific origins (localhost:3000, 10.75.5.233:3000)

✓ Rate limiting on sensitive endpoints (upload: 10/hr, recommend: 30/min, chat: 20/min)

✓ File validation: PDF magic bytes check + 5MB size limit before parsing

✓ HttpOnly cookies for JWT (prevents XSS token theft)

✓ SameSite=Lax cookie policy (CSRF mitigation)

✓ Flask-Compress for response compression (reduces attack surface for traffic interception)

✓ Backend input validation on all endpoints before DB operations

✓ UNIQUE constraint on `kanban_cards(user_id, job_id)` prevents duplicate saves

⚠ Security Gap – Exposed Secrets in .env:

The `backend/.env` file contains live API keys (Groq, OpenRouter, Grok), Gmail credentials, and the JWT secret committed to the repository. For production deployment, these must be:

- Rotated immediately (all listed API keys)
- Stored in a proper secrets manager (AWS Secrets Manager, GitHub Secrets, etc.)
- Added to `.gitignore` (currently listed there but file may still be tracked)

9. How to Run the Project Locally

Prerequisites

Tool	Version	Purpose
Node.js	≥ 20.x	Frontend runtime
npm	≥ 10.x	Package manager
Python	≥ 3.10	Backend runtime
pip	Latest	Python package manager
Google Chrome / Firefox	Any	For testing the UI

Step 1 – Clone & Configure Environment

```
git clone <repository-url>
cd "minor project Copy"

# Copy environment templates
cp .env.local.example .env.local
cp backend/.env.example backend/.env
```

Step 2 – Configure Environment Variables

Frontend — edit `.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:5000
NEXT_PUBLIC_GOOGLE_CLIENT_ID=<your-google-oauth-client-id>
```

Backend — edit `backend/.env`:

```
GOOGLE_CLIENT_ID=<your-google-oauth-client-id>
JWT_SECRET=<generate-a-random-256-bit-secret>
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=<your-gmail-address>
SMTP_PASS=<your-gmail-app-password>
GROQ_API_KEY=<your-groq-api-key>
```

```
FRONTEND_URL=http://localhost:3000
FLASK_ENV=development
SCRAPER_ENABLED=false
```

Step 3 – Install Backend Dependencies

```
cd backend

# Create virtual environment
python -m venv venv

# Activate virtual environment
# On Windows:
venv\Scripts\activate
# On macOS/Linux:
source venv/bin/activate

# Install Python packages
pip install -r requirements.txt
```

Step 4 – Start the Backend Server

```
# Still in backend/ with venv activated
python app.py

# Expected output:
# * Running on http://127.0.0.1:5000
# * BERT model loaded: all-MiniLM-L6-v2
# * Job embeddings pre-computed for 10,000 jobs
```

Note: First run downloads the BERT model (~80MB). Subsequent starts are instant due to caching in `.venv/` .

Step 5 – Install & Start Frontend

```
# In the root directory (not backend/)
npm install

# Start development server with Turbopack
npm run dev

# Expected output:
# ▲ Next.js 16.2.3 (Turbopack)
# - Local:    http://localhost:3000
```

Step 6 – Access the Application

Service	URL
Frontend (Next.js)	`http://localhost:3000`
Backend API (Flask)	`http://localhost:5000`
API Health Check	`http://localhost:5000/health`

Running Tests

```
# Frontend unit tests
npm run test

# Backend tests (in backend/ with venv activated)
pytest tests/ -v

# Backend tests with coverage
pytest tests/ --cov=. --cov-report=html
```

Production Build

```
# Build Next.js for production
npm run build

# Start production server
npm run start
```

10. Gaps, Limitations & Future Improvements

Current Limitations

Area	Gap / Issue	Severity
Database	SQLite is a file-based single-writer database. Does not support concurrent writes in multi-user production deployments. Should be replaced with PostgreSQL.	High
Security	API keys (Groq, JWT secret, Gmail credentials) are present in .env file potentially committed to version control. Must be rotated and stored in a secrets manager.	Critical
Live Job Scraping	LinkedIn/Indeed/Internshala scraping violates their Terms of Service and is fragile (breaks when site structure changes). Not production-suitable.	Medium
Resume Storage	Resume text stored as raw TEXT in SQLite. No file storage (S3/GCS). Only one resume per user (UNIQUE constraint on user_id).	Medium
ML Model	Skill extraction is regex-based with a fixed list of ~70 skills. Does not generalize to domain-specific or emerging skills. Should be replaced with NER (spaCy/Flair).	Medium
Authentication	JWT secret is static. No token rotation/refresh mechanism. Compromised tokens remain valid for 7 days.	Medium
Real-Time	No WebSocket support. Notifications require page refresh. Kanban board doesn't reflect multi-device updates in real time.	Low
Job Data Freshness	Static jobs.json (~10K jobs) has no update mechanism. Job postings can be stale/closed with no indication to the user.	Low

Mobile UX	Kanban drag-and-drop may not work well on touch devices. Charts can overflow on very small screens.	Low
No Admin Panel	No admin interface to manage users, moderate content, or view platform-wide analytics.	Low

Recommended Future Improvements

Infrastructure

- Migrate from SQLite → PostgreSQL (concurrent users)
- Containerize with Docker + docker-compose
- Deploy backend to Render / Railway / AWS EC2
- Deploy frontend to Vercel
- Store secrets in AWS Secrets Manager or Vault
- Add Redis for caching and session storage

Features

- Real-time notifications with WebSocket (Socket.io)
- Multiple resume versions per user
- Job application status email reminders
- Admin dashboard for platform management
- LinkedIn/GitHub profile import
- Collaborative team hiring for enterprise recruiters

ML / AI Improvements

- Replace regex skill extraction with spaCy NER
- Fine-tune BERT on Indian job market data
- Add learning path recommendations (Coursera/Udemy API)
- Salary prediction model
- Interview question generation from JD
- Resume quality score and improvement suggestions

Security Hardening

- JWT refresh token rotation mechanism
- Two-factor authentication (TOTP)
- Replace live scraping with official job APIs (RapidAPI, Adzuna)
- Add HTTPS/TLS certificates
- Input sanitization for XSS prevention
- Penetration testing before production launch

Project Maturity Assessment

Aspect	Status	Notes

Core AI/ML Functionality	Complete	BERT matching, skill extraction, ranking all working
Frontend UI/UX	Complete	Responsive, dark mode, all pages implemented
Authentication	Complete	Email + Google OAuth, role-based access
Database Schema	Complete	6 tables, migrations, proper relations
API Coverage	Complete	30+ endpoints, rate limiting, error handling
Testing	Partial	Backend tests exist; frontend coverage incomplete
Production Readiness	Partial	SQLite + exposed secrets block production use
Documentation	Partial	README exists; inline code docs are sparse
Live Job Data	Partial	Scraper works but ToS-violating; static data is stale
Monitoring / Logging	Missing	No structured logging, error tracking, or uptime monitoring

Summary: CareerBuilder is a fully-functional, full-stack AI job platform that demonstrates strong engineering breadth — NLP/ML integration, dual-sided marketplace design, OAuth, real-time-like Kanban, and AI chatbot. For academic purposes it is complete and impressive. Moving to production requires a database upgrade, secrets rotation, and replacing the job scraper with legitimate data sources.

CareerBuilder – Minor Project Report · 6th Semester Computer Science & Engineering · May 2026
Generated by Claude Code · Anthropic